

Appunti programmazione funzionale

Funzioni utili

List.sort: ('a -> 'a -> int) -> 'a list -> 'a list (nel compare ritorni -1 quello che non vuoi che cambi 0 se sono uguali e 1 se vuoi che si invertano)

List.iter: ('a -> unit) -> 'a list -> unit

Iterators

List.map: ('a -> 'b) -> 'a list -> 'b list

List scanning

List.for_all: ('a -> bool) -> 'a list -> bool

List.exists: ('a -> bool) -> 'a list -> bool

List searching

List.find : ('a -> bool) -> 'a list -> 'a

List.filter: ('a -> bool) -> 'a list -> 'a list

Altre utili sulle liste

List.rev_map : ('a -> 'b) -> 'a list -> 'b list

List.find: ('a -> bool) -> 'a list -> 'a

List.partition: ('a -> bool) -> 'a list -> 'a list * 'a list

List.fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a

List.fold_right : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a

List.cons y = function x -> y::x

Lettura da input

read_line() per leggere una singola linea (stringa), per la fine basta la stringa vuota

read_int() per leggere un singolo intero, per la fine bisogna usare il try

Unioni

@ concatenazione ([3;3;3]@[4;4;4] = [3;3;3;4;4;4])

:: cons prende un elemento e lo aggiunge in testa alla lista

Stringhe

String.make n x (* Make ti da una stringa che ripete x n volte *)

^ per concatenare due stringhe

nomestringa.[i] per prendere il suo i-esimo elemento

Currificazione di funzioni

allora, una funzione f_c si dice la *forma currificata* di f se e solo se:

1. il tipo di f_c è il seguente

$$f_c : \alpha_1 \rightarrow (\alpha_2 \rightarrow \dots \rightarrow (\alpha_n \rightarrow \alpha) \dots)$$

2. per ogni $a_1, \dots, a_n$, si ha che

$$f(a_1, \dots, a_n) = (((f_c a_1) a_2) \dots a_n)$$

Codici

Backtracking

Search_subset

Dato un set rappresentato da una lista e un numero n ritorna un sottoinsieme di S la cui somma da n

```
int list -> int
let sumof list =
  let rec aux risultato list =
    match list with
    | [] -> risultato
    | x::rest -> aux (risultato+x) rest
  in aux 0 list;;

exception NessunInsiemeTrovato
int -> int list -> int list
let subset_search n s =
  let rec aux solution altri =
    if sumof solution = n then solution
    else if sumof solution > n then raise NessunInsiemeTrovato
    else
      match altri with
      | [] -> raise NessunInsiemeTrovato
      | x::rest -> try
          aux (x::solution) rest
        with NessunInsiemeTrovato -> aux solution rest
  in aux [] s;;
```

Problema delle n regine

Trova una disposizione in una scacchiera nxn di n regine senza che nessuna sia in conflitto

```
(* Problema delle N regine.
L'indice dell'array rappresenta la colonna, il valore la riga

Creiamo una funzione scacco con due coppie di interi che ci dice se la prima posizione è sotto scacco della seconda
Per vedere se è sotto scacco se i-m = j-n -> stessa diagonale discendente
| | | | | | | | | | i+j = m+n -> stessa diagonale ascendente *)

(* scacco : int * int -> int * int -> bool *)
(* scacco (i,j) (m,n) = true se due regine, posizionate sulle caselle
(i,j) e (m,n) sono sotto scacco reciproco *)
int * int -> int * int -> bool
let scacco (i,j) (m,n) =
(i=m) || (i+j=m+n) || (i-j = m-n)
(* Ciascuna regina grazie alla rappresentazione non può trovarsi sulla stessa colonna *)

(* Creiamo una funzione che mi dice se quella posizione è salva o meno. Serve a aggiungere una nuova regina nella riga m sulla colonna nuova *)
(*Dobbiamo ora definire una funzione che, data la rappresentazione di una scacchiera
in cui sono state posizionate m regine e un intero k, determini se la m+1-esima regina
si può mettere nella riga k. *)
(* Serve a associare la colonna alla riga *)
(* combine : 'a list -> 'b list -> ('a * 'b) list *)
(* combine [x1;...;xn] [y1;...;yn] = [(x1,y1);...;(xn,yn)] *)
```

```
'a list -> 'b list -> ('a * 'b) list
let rec combine lst1 lst2 =
  match (lst1,lst2) with
  | ([],[]) -> []
  | (x::rest1,y::rest2) -> (x,y)::combine rest1 rest2
  | _ -> failwith "combine"

int -> int -> int list
let upTo n m =
  let rec aux result n m =
    if n>m then result
    else
      aux (n::result) (n+1) m
  in aux [] n m;;

(* safe : int list -> int -> bool *)
(* safe board k = true se la prossima regina si puo' mettere alla
riga k
(* aux: (int * int) list -> bool *)
(* funzione ausiliaria che controlla tutte le caselle della lista *)
(* aux lista_caselle = true se nessuna casella in lista_caselle mette
sotto scacco una regina nella casella
(m+1,k), dove m e' la lunghezza di board *)*)
```

```
int list -> int -> bool
let safe board k =
  let m = List.length board in
  let rec aux = function
  [] -> true
  | (i,j)::rest ->
    not (scacco (i,j) (k,m+1)) && aux rest
  in aux (List.combine board (upTo 1 m))

(* Adesso scrivo una funzione search che mi trovi almeno una soluzione *)
exception NotFound
(* Ci manca l'informazione sulla colonna da inserire nel with *)
(* queens n = soluzione al problema delle n regine *)
(* search : int -> int -> int list -> int list *)
(* search c r board = soluzione al problema delle n regine
che estende la scacchiera board dove sono gia' state
sistemate c-1 regine, e con la posizione della c-esima
regina in una riga maggiore o uguale a r *)
```

```
int -> int list
let queens n =
  let rec search c r board =
    if c > n then board
    else
      if r > n then raise NotFound
      else
        if safe board r
        then
          try search (c+1) 1 (board@[r])
          with NotFound -> search c (r+1) board
        else search c (r+1) board
    in search 1 1 []
```

Le tre

```
(* interleave: 'a-> 'a list-> 'a list list, tale che interleave
x lst riporti una lista con tutte le liste che si ottengono inserendo x in
qualsiasi posizione in lst. Utilizzare la funzione List.map.
Ad esempio, interleave 10 [0;1;2] = [[10; 0; 1; 2]; [0; 10; 1;
2]; [0; 1; 10; 2]; [0; 1; 2; 10]]. *)

'a list -> 'a list list
let rec interleave x lst = match lst with
| [] -> [[x]]
| y::rest -> (x::lst) :: List.map( function z-> y::z) (interleave x rest);;

(*permut: 'a list-> 'a list list, tale che permut lst riporti una
lista con tutte le permutazioni di lst (in qualsiasi ordine).*)
'a list -> 'a list list
let rec permut list =
  match list with
  | [] -> [[]]
  | x::rest -> List.flatten(List.map (interleave x) (permut rest));;

(* Applicata a una lista list rappresentante un insieme S, riporta una lista con tutti i sottoinsiemi di S *)

'a list -> 'a list list
let rec powerset list =
  match list with
  | [] -> [[]]
  | x::rest -> let prest = powerset rest
  in prest @ (List.map (List.cons x) prest);;
```

Merge sort

```
(* Merge sort*)
'a list -> 'a list -> 'a list
let rec merge l1 l2 =
  match (l1,l2) with
  | ([],l) | (l,[]) -> l
  | (x::xs,y::ys) -> if x<=y then x::(merge xs (y::ys))
  else y::(merge (x::xs) ys)
;;

'a list -> 'a list * 'a list
let split l =
  let rec split_aux l1 l2 = function
  | [] -> (l1,l2)
  | [x] -> (x::l1,l2)
  | x::y::xs -> split_aux (x::l1) (y::l2) xs
  in split_aux [] [] l
  ;;

'a list -> 'a list
let rec mergesort = function
| [] -> []
| [x: 'a] -> [x]
| l: 'a list -> let (xs,ys) = split l
  in merge (mergesort xs) (mergesort ys)
;;
```